

SQL DATA ANALYSIS PROJECT REPORT

PROJECT TITLE

Supply Chain Database Analysis Using SQL

PROJECT OVERVIEW

This project presents a structured SQL analysis of a real-world Supply Chain dataset, hosted in a PostgreSQL database. A total of 15 queries were written and executed against the supply_chain table, organized into five analytical themes covering Sales & Revenue, Inventory Analysis, Logistics Analysis, Supplier Analysis, and Cost & Profit.

Each query demonstrates practical SQL techniques such as aggregation, grouping, filtering, sorting, and derived calculations to answer specific business questions about product performance, stock levels, shipping efficiency, and supplier reliability.

Every query is documented with the exact SQL statement, a screenshot of its execution output, and a concise explanation of the result, illustrating how relational database querying can support data-driven decision making across a supply chain operation.

Prepared By	Arjun Chikane
Report Type	SQL Data Analysis Portfolio Project
Database	PostgreSQL

SQL CONCEPTS DEMONSTRATED

SELECT & FROM	Aggregate Functions (SUM, AVG)
GROUP BY	ORDER BY & Sorting
WHERE Filtering	LIMIT Clauses
Derived Columns	Multi-Column Aggregation
Business Insights	Business Recommendations

PURPOSE

This project demonstrates practical SQL querying skills by using aggregation, grouping, filtering, and derived calculations to solve real-world supply chain problems. It reflects the ability to translate raw relational data into actionable, business-relevant insights that support informed, data-driven decision making.

Report Overview

This report documents 15 SQL queries written against the supply_chain PostgreSQL table, grouped into five analytical themes. Each page presents the exact query, a screenshot of its execution output, and a short explanation of what the query does and what its result shows.

15

TOTAL QUERIES

5

ANALYSIS THEMES

1

SOURCE TABLE

PostgreSQL

DATABASE

CONTENTS

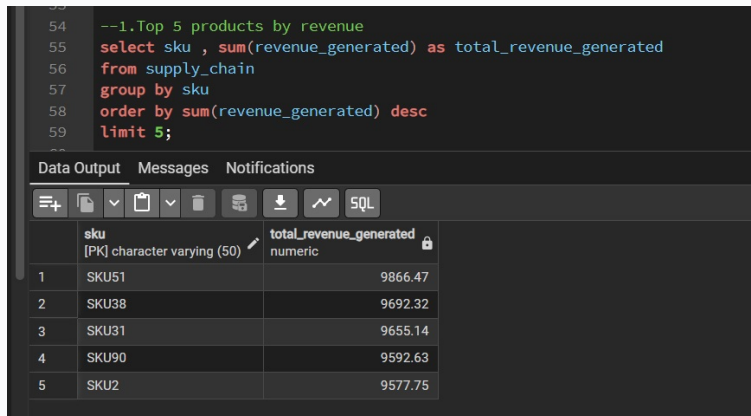
Sales & Revenue	3 queries
Inventory Analysis	3 queries
Logistics Analysis	3 queries
Supplier Analysis	3 queries
Cost & Profit	3 queries

Top 5 Products by Revenue

SQL QUERY

```
select sku, sum(revenue_generated) as total_revenue_generated
from supply_chain
group by sku
order by sum(revenue_generated) desc
limit 5;
```

QUERY OUTPUT



The screenshot shows a SQL query editor with a dark theme. The query is: `--1.Top 5 products by revenue`
`select sku , sum(revenue_generated) as total_revenue_generated`
`from supply_chain`
`group by sku`
`order by sum(revenue_generated) desc`
`limit 5;`

Below the query, there is a tabbed interface with 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with 2 columns: 'sku' and 'total_revenue_generated'. The table has 5 rows of data.

	sku [PK] character varying (50)	total_revenue_generated numeric
1	SKU51	9866.47
2	SKU38	9692.32
3	SKU31	9655.14
4	SKU90	9592.63
5	SKU2	9577.75

EXPLANATION

Ranks individual SKUs by total revenue generated and returns the top 5 performers. SKU51 leads with \$9,866.47, closely followed by SKU38, SKU31, SKU90 and SKU2 — all clustered within about \$300 of each other, showing strong revenue is spread across several products rather than dominated by one.

Average Revenue per Product Type

SQL QUERY

```
select product_type, avg(revenue_generated) as average_revenue_generated
from supply_chain
group by product_type
order by avg(revenue_generated) desc;
```

QUERY OUTPUT

61
62 --2.Average revenue per product type
63 select product_type , avg(revenue_generated) as average_revenue_generated
64 from supply_chain
65 group by product_type
66 order by avg(revenue_generated) desc;
67

Data Output Messages Notifications

SQL

	product_type character varying (50)	average_revenue_generated numeric
1	cosmetics	6212.3565384615384615
2	skincare	6040.7042500000000000
3	haircare	5131.0417647058823529

EXPLANATION

Groups records by product category and calculates the average revenue generated per category. Cosmetics has the highest average revenue (\$6,212.36), ahead of Skincare (\$6,040.70) and Haircare (\$5,131.04), indicating cosmetics items generate more revenue per unit on average.

Supply Chain SQL Report — Sales & Revenue

Page 4

Product Type Contributing Highest Revenue

SQL QUERY

```
select product_type, sum(revenue_generated) as highest_revenue_generated
from supply_chain
group by product_type
order by sum(revenue_generated) desc;
```

QUERY OUTPUT

```
67
68 --3.Product type contributing highest revenue
69 select product_type , sum(revenue_generated) as highest_revenue_generated
70 from supply_chain
71 group by product_type
72 order by sum(revenue_generated) desc;
```

Data Output Messages Notifications

SQL

	product_type character varying (50)	highest_revenue_generated numeric
1	skincare	241628.17
2	haircare	174455.42
3	cosmetics	161521.27

EXPLANATION

Sums total revenue by product category rather than averaging it, to see overall contribution. Skincare contributes the most total revenue (\$241,628.17), followed by Haircare (\$174,455.42) and Cosmetics (\$161,521.27) — the opposite order from the average-revenue result, meaning Skincare's volume of sales outweighs its lower per-unit average.

```
SELECT sku, product_type, stock_levels, number_of_products_sold
FROM supply_chain
WHERE stock_levels < 50
AND number_of_products_sold > 500;
```

```
75 /*Inventory Analysis
```

```
76 Products with low stock but high sales
```

```
77 Total stock per product type
```

```
78 Products with highest order quantity*/
```

```
79
```

```
80
```

```
81 --1.Products with low stock but high sales
```

```
82 SELECT sku, product_type, stock_levels, number_of_products_sold
```

```
83 FROM supply_chain
```

```
84 WHERE stock_levels < 50
```

```
85 AND number_of_products_sold > 500;
```

```
86
```

Data Output Messages Notifications

SQL

	product_type character varying (50)	highest_revenue_generated numeric
1	skincare	241628.17
2	haircare	174455.42
3	cosmetics	161521.27

Filters for products that combine low remaining stock (under 50 units) with high sales volume (over 500 units sold) — a signal of products at risk of stockouts. Note: the captured screenshot for this query still shows the previous query's result panel (product type revenue totals), so no filtered rows are visible here — this query should be re-run and re-captured to confirm whether any products meet both conditions.

Total Stock per Product Type

SQL QUERY

```
select product_type, sum(stock_levels) as total_stock
from supply_chain
group by product_type
order by total_stock desc;
```

QUERY OUTPUT

```
87 --2.Total stock per product type
88 select product_type , sum(stock_levels) as total_stock
89 from supply_chain
90 group by product_type
91 order by total_stock desc;
```

Data OutputMessagesNotifications

SQL

	product_type character varying (50)	total_stock bigint
1	haircare	1644
2	skincare	1608
3	cosmetics	1525

EXPLANATION

Aggregates total stock units held across all products within each category. Haircare holds the most stock (1,644 units), just ahead of Skincare (1,608) and Cosmetics (1,525), showing inventory is fairly evenly distributed across categories.

Products with Highest Order Quantity

SQL QUERY

```
select * from supply_chain
order by order_quantities desc
limit 5;
```

QUERY OUTPUT

```
--3.Products with highest order quantity
select * from supply_chain
order by order_quantities desc
limit 5;
```

Data OutputMessagesNotifications

SQL

	sku [PK] character varying (50)	product_type character varying (50)	price numeric (10,2)	availability integer
1	SKU0	haircare	69.81	55
2	SKU90	skincare	13.88	56
3	SKU33	cosmetics	64.80	63
4	SKU19	skincare	51.12	100
5	SKU2	haircare	11.32	34

EXPLANATION

Returns the full record for the 5 products with the largest order quantities, giving a complete profile (price, availability, category, etc.) of the highest-demand items for further investigation.

Average Shipping Cost by Transport Mode

SQL QUERY

```
select transportation_modes, avg(shipping_costs) as avg_shipping_costs
from supply_chain
group by transportation_modes;
```

QUERY OUTPUT

100

/*Logistics Analysis

101

Average shipping cost by transport mode

102

Fastest shipping mode

103

Most expensive shipping routes*/

104

105

--1.Average shipping cost by transport mode

106

select transportation_modes , avg(shipping_costs) as avg_shipping_costs

107

from supply_chain

108

group by transportation_modes;

109

Data Output

Messages

Notifications

transportation_modes

avg_shipping_costs

character varying (50)

numeric

1

Air

6.0176923076923077

2

Sea

4.9700000000000000

3

Rail

5.4692857142857143

4

Road

5.5424137931034483

EXPLANATION

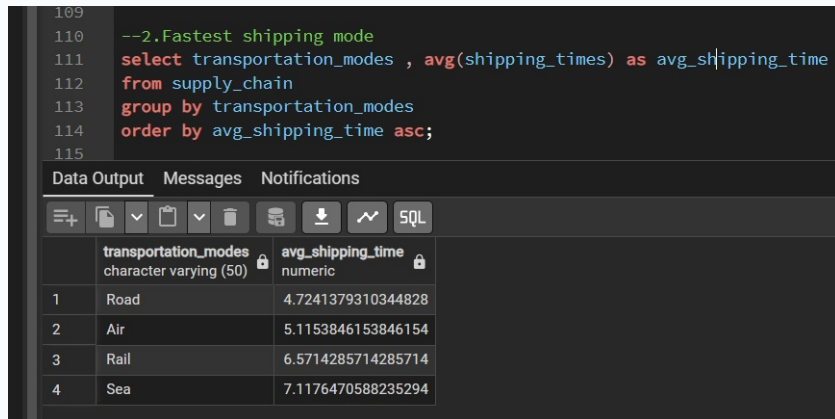
Compares average shipping cost across transportation modes. Air is the most expensive at \$6.02 per shipment, while Sea is the cheapest at \$4.97 — Rail (\$5.47) and Road (\$5.54) fall in between.

Fastest Shipping Mode

SQL QUERY

```
select transportation_modes, avg(shipping_times) as avg_shipping_time
from supply_chain
group by transportation_modes
order by avg_shipping_time asc;
```

QUERY OUTPUT



The screenshot shows a SQL query execution interface. At the top, the query is displayed in a dark-themed editor. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'transportation_modes' and 'avg_shipping_time'. The table contains four rows of data, ordered by average shipping time from fastest to slowest.

	transportation_modes	avg_shipping_time
1	Road	4.7241379310344828
2	Air	5.1153846153846154
3	Rail	6.5714285714285714
4	Sea	7.1176470588235294

EXPLANATION

Orders transportation modes by average shipping time to identify the fastest option. Road is fastest (4.72 days average), followed by Air (5.12), Rail (6.57), and Sea, which is the slowest (7.12 days).

Most Expensive Shipping Routes

SQL QUERY

```
select routes, sum(shipping_costs) as total_shipping_cost
from supply_chain
group by routes
order by total_shipping_cost desc;
```

QUERY OUTPUT

116 --3. Most expensive shipping routes
117 select routes , sum(shipping_costs) as total_shipping_cost
118 from supply_chain
119 group by routes
120 order by total_shipping_cost desc;
121

Data Output Messages Notifications

SQL

	routes character varying (50)	total_shipping_cost numeric
1	Route A	231.31
2	Route B	205.43
3	Route C	118.08

EXPLANATION

Sums total shipping cost incurred per route. Route A has accumulated the highest total shipping cost (\$231.31), ahead of Route B (\$205.43) and Route C (\$118.08), likely reflecting higher shipment volume along that route.

Supply Chain SQL Report — Logistics Analysis

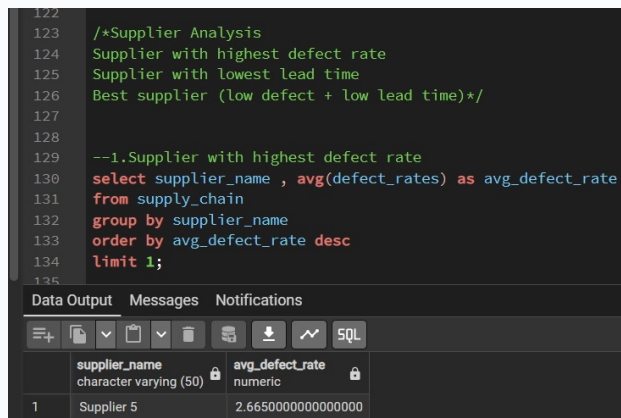
Page 11

Supplier with Highest Defect Rate

SQL QUERY

```
select supplier_name, avg(defect_rates) as avg_defect_rate
from supply_chain
group by supplier_name
order by avg_defect_rate desc
limit 1;
```

QUERY OUTPUT



The screenshot shows a SQL query editor with a dark background. The query is as follows:

```
122
123 /*Supplier Analysis
124 Supplier with highest defect rate
125 Supplier with lowest lead time
126 Best supplier (low defect + low lead time)*/
127
128
129 --1.Supplier with highest defect rate
130 select supplier_name , avg(defect_rates) as avg_defect_rate
131 from supply_chain
132 group by supplier_name
133 order by avg_defect_rate desc
134 limit 1;
135
```

Below the query, there is a tabbed interface with 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the following data:

	supplier_name character varying (50)	avg_defect_rate numeric
1	Supplier 5	2.6650000000000000

EXPLANATION

Identifies the single supplier with the worst average defect rate. Supplier 5 has the highest average defect rate at 2.665%, flagging it as the weakest performer on quality among all suppliers.

Supplier with Lowest Lead Time

SQL QUERY

```
select supplier_name, avg(lead_time) as supplier_lowest_lead_time
from supply_chain
group by supplier_name
order by avg(lead_time) asc
limit 1;
```

QUERY OUTPUT

135

136

137

138

139

140

141

--2.Supplier with lowest lead time
select supplier_name , avg(lead_time) as supplier_lowest_lead_time
from supply_chain
group by supplier_name
order by avg(lead_time) asc
limit 1;

Data Output

Messages

Notifications

supplier_name

character varying (50)

supplier_lowest_lead_time

numeric

1

Supplier 1

14.77777777777778

EXPLANATION

Identifies the supplier with the fastest average lead time. Supplier 1 delivers fastest, with an average lead time of 14.78 days, making it the top choice when speed of delivery matters most.

```
select
    supplier_name,
    avg(defect_rates) as avg_defect_rate,
    avg(lead_time) as avg_lead_time
from supply_chain
group by supplier_name
order by avg_defect_rate asc, avg_lead_time asc
limit 5;
```

```

143 --3.Best supplier (low defect + low lead time)
144 select
145     supplier_name,
146     avg(defect_rates) as avg_defect_rate,
147     avg(lead_time) as avg_lead_time
148 from supply_chain
149 group by supplier_name
150 order by avg_defect_rate asc, avg_lead_time asc
151 limit 5;
152

```

Data Output Messages Notifications			
≡ 📄 ▼ 📄 🗑️ 🗑️ 📄 📄 📄 📄 SQL			
	supplier_name character varying (50)	avg_defect_rate numeric	avg_lead_time numeric
1	Supplier 1	1.8033333333333333	14.777777777777778
2	Supplier 4	2.3388888888888889	15.222222222222222
3	Supplier 2	2.3618181818181818	18.545454545454545
4	Supplier 3	2.4660000000000000	20.133333333333333
5	Supplier 5	2.6650000000000000	18.055555555555556

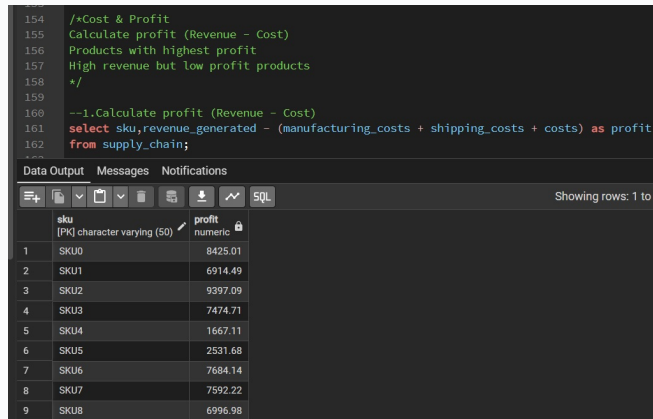
Ranks all suppliers jointly by defect rate and lead time to find the best all-round performer. Supplier 1 ranks first on both dimensions (1.80% defect rate, 14.78-day lead time), confirming it as the strongest overall partner, ahead of Supplier 4 and Supplier 2.

Calculate Profit (Revenue – Cost)

SQL QUERY

```
select sku, revenue_generated - (manufacturing_costs + shipping_costs + costs) as  
profit  
from supply_chain;
```

QUERY OUTPUT



The screenshot shows a SQL IDE interface. The top pane displays the SQL query with comments: `/*Cost & Profit
Calculate profit (Revenue - Cost)
Products with highest profit
High revenue but low profit products
*/
--1.Calculate profit (Revenue - Cost)
select sku, revenue_generated - (manufacturing_costs + shipping_costs + costs) as profit
from supply_chain;`. The bottom pane shows the query results in a table with two columns: 'sku' (character varying (50)) and 'profit' (numeric). The table contains 9 rows of data, with SKUs ranging from SKU0 to SKU8 and profit values ranging from 8425.01 to 6996.98.

	sku [PK] character varying (50)	profit numeric
1	SKU0	8425.01
2	SKU1	6914.49
3	SKU2	9397.09
4	SKU3	7474.71
5	SKU4	1667.11
6	SKU5	2531.68
7	SKU6	7684.14
8	SKU7	7592.22
9	SKU8	6996.98

EXPLANATION

Computes profit per SKU by subtracting manufacturing cost, shipping cost, and other costs from revenue generated. Results range widely per product — for example SKU2 shows \$9,397.09 profit while SKU4 shows only \$1,667.11 — highlighting large profitability differences between individual products.

Products with Highest Profit

SQL QUERY

```
select sku, revenue_generated - (manufacturing_costs + shipping_costs + costs) as
profit
from supply_chain
order by profit desc
limit 5;
```

QUERY OUTPUT

164

--2.Products with highest profit

165

select sku,revenue_generated - (manufacturing_costs + shipping_costs + costs) as profit

166

from supply_chain

167

order by profit desc

168

limit 5;

169

Data Output

Messages

Notifications

Showing rows: 1 to

	sku [PK] character varying (50)	profit numeric
1	SKU2	9397.09
2	SKU38	9337.48
3	SKU67	9288.32
4	SKU88	9268.84
5	SKU51	9152.52

EXPLANATION

Sorts the profit calculation to surface the 5 most profitable individual products. SKU2 tops the list at \$9,397.09, followed closely by SKU38, SKU67, SKU88, and SKU51, all above \$9,150 in profit.

High Revenue but Low Profit Products

SQL QUERY

```
select product_type, avg(revenue_generated) as avg_revenue,  
avg(revenue_generated - (manufacturing_costs + shipping_costs + costs)) as avg_profit  
from supply_chain  
group by product_type  
order by avg(revenue_generated) desc, avg_profit asc;
```

QUERY OUTPUT

170 --3.High revenue but low profit products
171 select product_type , avg(revenue_generated) as avg_revenue,
172 avg(revenue_generated - (manufacturing_costs + shipping_costs + costs)) as avg_profit
173 from supply_chain
174 group by product_type
175 order by avg(revenue_generated) desc , avg_profit asc;
176

Data Output Messages Notifications

Showing rows:

	product_type character varying (50)	avg_revenue numeric	avg_profit numeric
1	cosmetics	6212.3565384615384615	5649.1515384615384615
2	skincare	6040.7042500000000000	5431.0685000000000000
3	haircare	5131.0417647058823529	4567.0044117647058824

EXPLANATION

Compares average revenue against average profit by category to spot categories that generate strong revenue but comparatively weaker profit. Cosmetics leads on both average revenue (\$6,212.36) and average profit (\$5,649.15), with Skincare and Haircare following in the same order — so no category currently shows a high-revenue/low-profit mismatch.

Supply Chain SQL Report — Cost & Profit

Page 17